

Documentação da Plataforma VNH

Hype App · Você no Hype

Arquitetura, dados, módulos, portal, comunicação e auditoria técnica

Versão 1.297.0 · atualizado em 3 de outubro de 2026

1. Visão geral
2. Arquitetura
3. Dados e acesso
4. Módulos do app interno
5. Portal do cliente, links externos e integrações
6. Como as partes se comunicam e como as funcionalidades se ligam
7. Auditoria técnica — 2026-10-03
8. Riscos e lacunas conhecidos

Visão geral

O que é

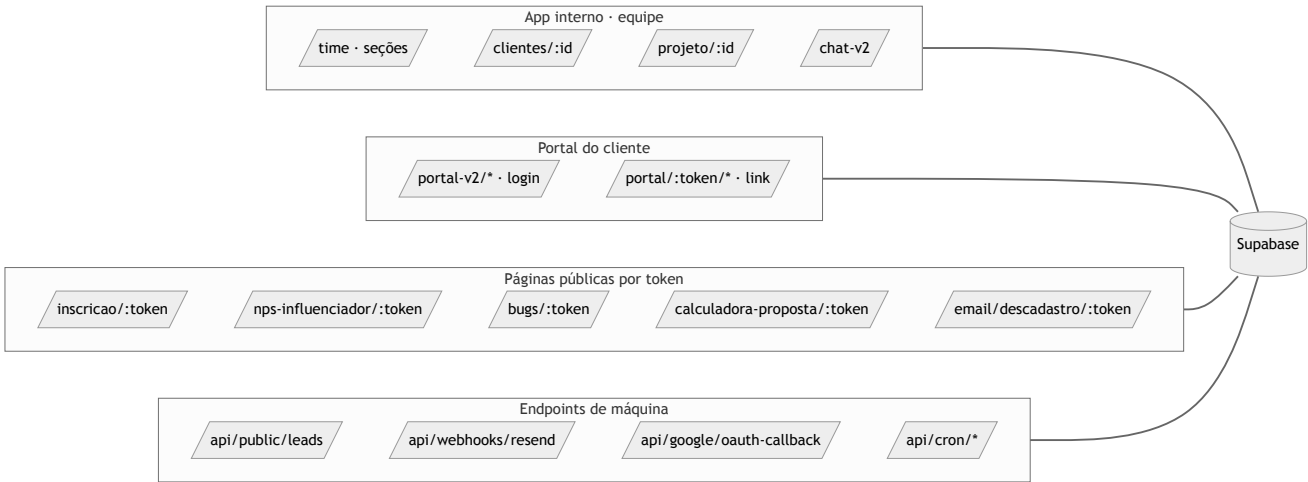
Plataforma de operação da agência **Você no Hype** (marketing de influência). Reúne, num só sistema, o que a equipe usa no dia a dia (comercial, clientes, campanhas, influenciadores, projetos e tarefas, reuniões, financeiro, metas, chat, time) e o que o **cliente** e o **influenciador** acessam por fora (portal, páginas públicas por link).

Construída no **Lovable** e conectada a um backend **Supabase** (Lovable Cloud). Produção em `https://plataforma.vocenhype.com.br` (Vercel).

Quem usa e por onde entra

Pessoa	Superfície	Como autentica
Equipe interna (admin ou membro com permissões)	App interno <code>/time</code> , <code>/clientes/\$id</code> , <code>/projeto/\$id</code> , <code>/chat-v2</code>	E-mail + senha (Supabase Auth), MFA se ativado
Cliente com login (empresa-cliente)	Portal do cliente V2 <code>/portal-v2/*</code>	E-mail + senha por convite; escopo por organização e por campanha
Cliente por link (sem login)	Portal por token <code>/portal/\$token/*</code>	Token público fixo do cliente
Influenciador / lead / terceiro	Páginas públicas: inscrição, NPS, bugs, proposta, descadastro	Token na URL
Sistemas externos	Webhooks e crons: leads (Make/Typeform), Resend, Google, Vercel Cron	Segredo/assinatura/Bearer

As cinco superfícies



Glossário

- Cliente** — empresa atendida pela agência. Cada cliente tem uma **organização** dedicada (para o portal).
- Campanha** — ação de influência de um cliente (briefing, orçamento, influenciadores, entregas, resultados).
- Influenciador (influ)** — criador de conteúdo; existe no **Banco de Influenciadores** e é vinculado a campanhas.
- Entrega** — conteúdo que o influenciador produz (roteiro → conteúdo → publicação), com versões e aprovação do cliente.
- Lead / oportunidade** — registro do Comercial (pipeline); vira Cliente + Projeto na conversão.
- Projeto** — trabalho interno (tarefas, marcos, bugs) ligado a um cliente.
- Hypito** — assistente da plataforma. Aparece em `hypito_*` (tabelas), `hypito_payload` nas mensagens do chat e em permissões. O serviço em si **não está neste repositório** [inferido: roda fora, usa as mesmas tabelas].
- Ambiente** — a organização em que a pessoa entra: `internal` (equipe) ou `client`.
- Seção** — módulo dentro de `/time` (`?section=`), não é rota separada.

Arquitetura

Stack

Camada	Tecnologia
Framework	TanStack Start (React 19 + TanStack Router, SSR via Nitro/h3), file-based routing
Build	Vite 8, TypeScript estrito, gerenciador bun (<code>bun.lock</code> , <code>bunfig.toml</code>)
UI	Tailwind v4, shadcn/ui (Radix, estilo "new-york"), lucide-react, TipTap (editor rico)
Dados	Supabase (Postgres + Auth + Realtime + Storage), <code>@supabase/supabase-js</code> , TanStack Query
Deploy	Vercel (<code>vercel.json</code> : headers de segurança + 2 crons); código sincroniza com Lovable pelo branch git
E-mail	Resend (provedor configurável por admin; webhook de entrega)
Outros	Web Push (VAPID), Google Calendar (OAuth), Open-Meteo (clima), WebRTC (chamadas, TURN opcional)

Regra de repositório (AGENTS.md): nunca reescrever histórico já publicado (force-push, rebase/amend/squash) — dessincroniza o Lovable.
Arquivos `src/integrations/supabase/*` são gerados.

Estrutura de pastas

<code>src/</code>	
<code> routes/</code>	rotas (file-based). <code>routeTree.gen.ts</code> é gerado
<code> components/</code>	telas e peças do app interno (uma <code>*Section</code> por módulo) + <code>ui/</code> (shadcn) + <code>shared/</code> (canônicos)
<code> features/client-portal-v2/</code>	o Portal V2 (layout, páginas, componentes, lib, testes)
<code> lib/</code>	regra de negócio, stores, server functions (<code>*.functions.ts</code>), engines puros
<code> hooks/</code>	<code>use-confirm</code> , <code>use-mobile</code> , <code>use-weather</code> , ...
<code> integrations/supabase/</code>	<code>client</code> , <code>client.server</code> (service-role), <code>auth-middleware</code> , <code>types</code> (gerados)
<code> styles.css</code>	<code>styles/</code> tokens (oklch) e utilidades
<code>supabase/migrations/</code>	144 arquivos SQL, timestamp-nomeados, aditivos
<code>public/</code>	PWA (manifest, sw.js sem cache), <code>version.json</code> , sons
<code>docs/</code>	documentação (ver docs/README.md)
<code>scripts/release.ts</code>	release local (não há CI)

Rotas

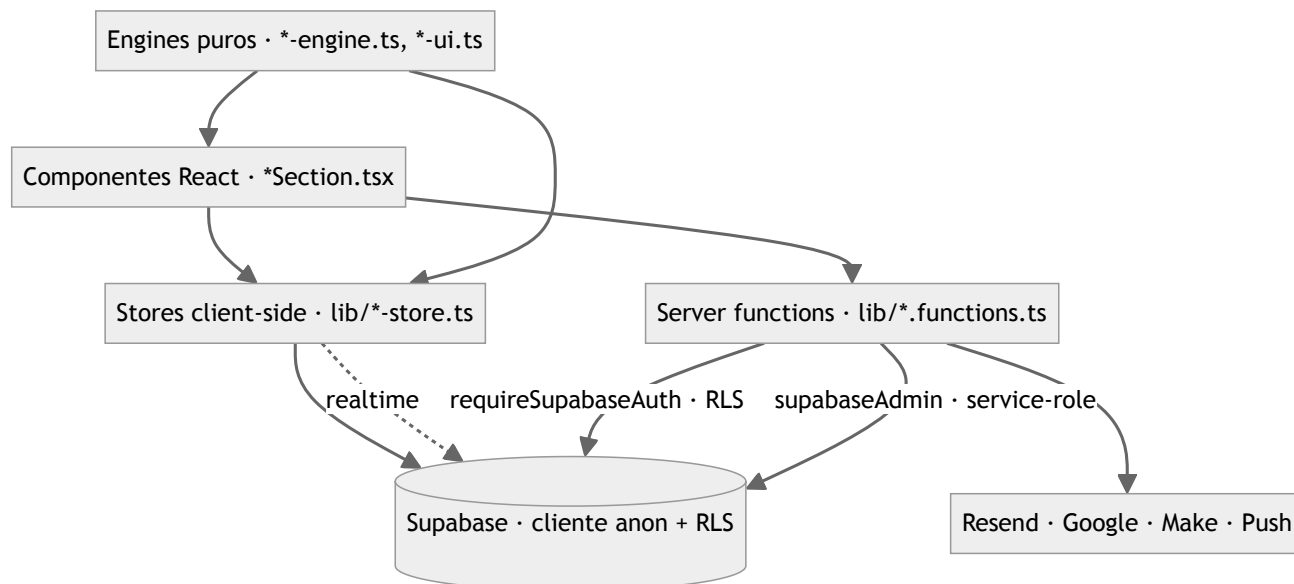
Layout autenticado `authenticated/route.tsx` : confere sessão (redireciona para `/`), `profiles.must_change_password` (→ `/primeiro-acesso`), inicializa stores/sync e a lista do time (poll 30s).

Rota	Tipo	Função
<code>/</code>	pública	Login (<code>signInWithPassword</code>)
<code>/authenticated/time</code>	app	Home do app: <code>AppShell</code> + seção por <code>?section=</code> (início, clientes, campanhas, projetos, reunioes, comercial, financeiro, time, influenciadores, metas, chat, configuracoes, problemas) e evento <code>nav:section</code>
<code>/clientes/\$id</code> , <code>/projeto/\$id</code>	app	Detalhe de cliente / projeto (únicas seções com rota própria, além do chat)
<code>/chat-v2</code> (+ <code>channel/\$id</code> , <code>dm/\$id</code> , <code>campaign/\$id</code>)	app	Chat V2 dentro do AppShell
<code>/design-system</code> , <code>/design-system-finance-concept</code>	app	Vitrine interna do design system
<code>/primeiro-acesso</code> , <code>/criar-senha</code> , <code>/redefinir-senha</code>	auth	Troca forçada (equipe), convite (cliente), recuperação
<code>/selecionar-ambiente</code> , <code>/acesso-pendente</code> , <code>/acesso-bloqueado</code>	auth	Escolha de organização, convite pendente, organização suspensa
<code>/portal-v2/*</code>	cliente	Portal com login (ver 05)
<code>/portal/\$token/*</code>	cliente	Portal por link (ver 05)
<code>/portal-app/*</code>	legado	Redirects para <code>/portal-v2</code>
<code>/inscricao/\$token</code> , <code>/nps-influenciador/\$token</code> , <code>/bugs/\$token</code> , <code>/calculadora-proposta/\$token</code> , <code>/email/descadastro/\$token</code>	pública	Páginas por token (ver 05)

Rota	Tipo	Função
<code>/api/public/leads</code> , <code>/api/webhooks/resend</code> , <code>/api/google/oauth-callback</code> , <code>/api/cron/email-flows</code> , <code>/api/cron/google-calendar-sync</code>	máquina	Endpoints (ver 05)
<code>/foco</code> , <code>/time-v2</code> , <code>/banco-influenciadores-v2</code>	redirect	Compatibilidade (cutover concluído)

Observação importante: a maior parte do app interno é **uma SPA dentro de** `/time` , trocada por `?section=` — não são rotas TanStack separadas. `src/lib/section-nav.ts` é a fonte dos tipos e rótulos de seção/aba.

Camadas



1. **UI** — uma `*Section.tsx` por módulo; subcomponentes na pasta do módulo.
2. **Engines/regra pura** — `entrega-engine` , `metas-engine` , `comercial-engine` , `performance-engine` , `insights-engine` , `aeo-engine` e `*-ui.ts` : sem I/O, testáveis; a UI nunca recalcula saúde/progresso na mão.
3. **Stores** — pub/sub escritos à mão (sem Redux/Zustand): `createTableArrayStore` (uma linha por entidade em `clientes` , `projetos` , `reunioes` , `financeiro_lancamentos` , `banco_influenciadores` , `marketing_tasks` , `metas` , `aeo_*` ...), mais `chat-store` , `workspace-store` , `shared-sync` (`shared_state`), `campanha-scoped-store` , `projeto-scoped-store` . Mantêm contrato síncrono `get/set/subscribe` ; `init()` roda uma vez no `beforeLoad` do layout autenticado.
4. **Server functions** (`createServerFn`) — 34 arquivos `*.functions.ts` . O token do usuário é anexado por `auth-attacher` e validado por `requireSupabaseAuth` (injeta `supabase` , `userId` , `claims`).
5. **Admin/service-role** — só em `*.server.ts` ou via `await import(...)` dentro de handlers; todo uso de `supabaseAdmin` passa por checagem explícita (`assertAdmin` / `is_admin()` ou validação de token).

Autenticação e ambientes

- Login por senha; `must_change_password` força `/primeiro-acesso` .
- **MFA** (TOTP) via `mfa.functions.ts` ; o portal e o app redirecionam para `/` se o nível de garantia exigir desafio.
- **Ambiente** (`user-environment.server.ts`): lê `organization_members` ativos em `organizations` ativas e devolve `internal` , `client` , `multiple` (→ `/selecionar-ambiente`), `pending` (→ `/acesso-pendente`) ou `suspended` (→ `/acesso-bloqueado`). Home do cliente: `/portal-v2/inicio` ; da equipe: `/time` .
- **Convites** (`organization-invites` , `accept-invite`): aceitam convites pendentes ao logar.
- **Recuperação de senha** com rate limit (`rate-limit.functions.ts`) antes de existir sessão.
- **Cofre de senhas** (`vault*.functions.ts`) com TOTP próprio e pedidos de acesso.

Comandos, variáveis de ambiente, deploy e convenções

Vivem em `../development/guia.md` (fonte única).

Dados e acesso

Backend: Supabase projeto `kehjxyzrolltsdaqzkww`. 144 migrations aditivas em `supabase/migrations/`. RLS ligada em todas as tabelas de domínio.

Modelo de dados por domínio

Domínio	Tabelas principais	Observação
Identidade e acesso	<code>profiles</code> , <code>user_roles</code> , <code>organizations</code> , <code>organization_members</code> , <code>campaign_members</code> , <code>access_audit_log</code> , <code>settings_audit_log</code> , <code>password_reset_requests</code> , <code>rate_limit_events</code>	<code>profiles.permissions</code> guarda as permissões do membro; <code>organizations.type</code> é <code>internal</code> ou <code>client</code>
Clientes e contratos	<code>clientes</code> , <code>contratos</code>	<code>clientes.data</code> é JSONB e embute as campanhas do cliente
Comercial	<code>leads</code> , <code>commercial_interactions</code> , <code>comercial_saved_views</code> , <code>pricing_settings</code>	Leads vêm do webhook ou da UI; <code>proposta</code> pública por token
Campanhas	<code>campanha_influenciadores</code> , <code>campanha_tarefas</code> , <code>campanha_documentos</code> , <code>campanha_cronograma</code> , <code>campanha_entregas</code> , <code>campanha_entrega_versoes</code> , <code>campanha_entrega_eventos</code> , <code>campanha_nps</code> , <code>campanha_nps_influenciador</code> , <code>campanha_influenciador_avaliacoes</code> , <code>campaign_cycles</code> , <code>inscricao_campanha_idempotency</code> , <code>influencer_approvals</code>	Tabelas "scoped": cada linha tem <code>campanha_id</code> ; o dado completo fica em <code>data</code> JSONB com <code>data.id</code> sincronizado por trigger
Influenciadores	<code>banco_influenciadores</code> , <code>projeto_influenciadores</code>	Dados bancários exigem permissão própria
Projetos e tarefas	<code>projetos</code> , <code>projeto_tarefas</code> , <code>task_blocks</code> , <code>task_dependencies</code> , <code>task_tags</code> , <code>marketing_tasks</code> , <code>marketing_standalone_tasks</code> , <code>time_entries</code>	Tarefas também existem dentro de campanhas e do Marketing; <code>task-aggregation</code> as une
Reuniões	<code>reunioes</code> , <code>reunioes_disponibilidade</code> , <code>google_calendar_connections</code> , <code>google_calendar_sync_state</code> , <code>google_oauth_states</code> , <code>shared_calendar_connection</code>	Sync opcional com Google Calendar
Financeiro	<code>financeiro_lancamentos</code> , <code>financeiro_recorrencias</code> , <code>financeiro_settings</code> , <code>financeiro_status_overrides</code>	Lançamentos podem apontar para cliente, campanha e influenciador
Metas	<code>metas</code> , <code>performance_events</code> , <code>performance_settings</code>	Objetivos e indicadores; motor de saúde puro em <code>metas-engine</code>
Chat	<code>chat_channels</code> , <code>chat_messages</code> , <code>chat_reads</code> , <code>chat_status</code> , <code>chat_conversations</code> , <code>chat_conversation_members</code> , <code>chat_message_attachments</code> , <code>chat_message_reactions</code> , <code>chat_pinned_messages</code> , <code>chat_saved_messages</code> , <code>chat_deliveries</code> , <code>chat_drafts</code>	Convive V1 (<code>/time?section=chat</code>) e V2 (<code>/chat-v2</code>); trigger sincroniza JSONB legado para tabelas
Marketing	<code>email_campaigns</code> , <code>email_campaign_steps</code> , <code>email_campaign_recipients</code> , <code>email_campaign_activity</code> , <code>email_flows</code> , <code>email_flow_enrollments</code> , <code>email_sends</code> , <code>email_templates</code> , <code>email_unsubscribes</code> , <code>email_provider_settings</code> , <code>aeo_prompts</code> , <code>aeo_rodadas</code> , <code>aeo_respostas</code> , <code>blog_comments</code> , <code>blog_likes</code>	E-mail (Resend), monitor AEO e blog
Central de Problemas	<code>bug_reports</code> , <code>bug_report_comments</code> , <code>bug_report_events</code> , <code>bug_report_attachments</code> , <code>bug_report_diagnostics</code>	Triagem exige <code>can_manage_problems</code>
Hypito	<code>hypito_action_log</code> , <code>hypito_alerts_sent</code> , <code>hypito_conversation_state</code> , <code>hypito_daily_briefing_runs</code> , <code>hypito_meeting_reminders_sent</code> , <code>hypito_pending_actions</code> , <code>hypito_reminders</code> , <code>hypito_report_runs</code> , <code>hypito_report_settings</code> , <code>hypito_user_prefs</code>	Assistente/automações
Plataforma	<code>workspace_settings</code> , <code>shared_state</code> , <code>platform_releases</code> , <code>outgoing_webhooks</code> , <code>webhook_settings</code> , <code>push_subscriptions</code> , <code>personal_reminders</code>	Config global e avisos de versão
Segurança	<code>vault_secret</code> , <code>vault_totp_secrets</code> , <code>vault_totp_attempts</code> , <code>vault_access_requests</code>	Cofre de senhas
Legado sem uso	<code>daily_game_sessions(+archive)</code> , <code>sudoku_daily_results</code> , <code>zip_daily_results</code>	Jogos removidos do app em 2026-10-03; tabelas e dados mantidos, nenhum código lê ou grava nelas

(`projeto_fases` foi removida na migration `20261003120000` .)

Storage (buckets)

`avatars`, `chat-attachments`, `bug-reports`, `task-attachments`, `financeiro-anexos`, `entrega-anexos`, `relatorios-mensais`, `aeo-evidencias` (5 receberam limite de 100 MB na remediação de setembro). Anexos públicos de bug-report por link vão em `public/<token>/...`. Relatórios mensais são entregues por URL assinada fresca (`getFreshRelatorioUrl*`).

Autorização em camadas

1. **RLS no banco** (a barreira real). Funções-chave: `is_admin()`, `has_permission(uid, perm)` (une admin + permissão), `can_manage_problems`, `is_internal_team_member`, `shares_internal_organization`, `is_active_client_admin_of`, `user_can_access_campanha`.
2. **Permissões de módulo** (`lib/permissions.ts`): `clientes`, `campanhas`, `contratos`, `projetos`, `reunioes`, `comercial`, `financeiro`, `time`, `influenciadores`, `influenciadores:bancario`, `metas`, `chat`, `senhas`, `configuracoes` (+ `:perfil/:workspace/:av/:senhas`), `membros`, `problemas`. Todas as tabelas de domínio chamam `has_permission` em suas policies.
3. **Server functions**: RLS-scoped (`requireSupabaseAuth`) por padrão; as que usam service-role validam admin ou token explicitamente.
4. **UI**: esconde/esmaece o que a pessoa não pode usar — conveniência, nunca a barreira.

Limitações aceitas e documentadas: - `clientes` aceita `clientes` **ou** `campanhas`, porque a campanha vive no JSONB do cliente. - `membros` é **decorativa**: criar/editar/excluir membro e redefinir senha seguem exigindo admin (um endpoint único altera `role`, e liberar abriria escalada de privilégio). - `senhas` é decorativa: o acesso real ao cofre é `configuracoes:senhas`.

Portal do cliente — isolamento

- Cliente com login: papéis `client_standard` (administra pessoas e acessos), `client_approver`, `client_viewer`; **nunca** pode alterar as próprias permissões (`prevent_client_permissions`); proteção contra remover o último admin; acesso restrito a campanhas liberadas (`campaign_members`, `user_can_access_campanha`).
- Cliente por token: o token público do cliente (`clientes.data.publicToken`) é a credencial; as funções do `cliente-link` validam o token e escopam tudo ao cliente.

Outras proteções

- **MFA/TOTP**: rate limit de login e recuperação; `X-Frame-Options`, `nosniff`, `Referrer-Policy` (Vercel).
- Webhook de leads com `X-Webhook-Secret` (segredo em `webhook_settings`, rotacionável por admin); Resend com assinatura Svix; crons com `Authorization: Bearer $CRON_SECRET`.
- Markdown do blog sanitizado (DOMPurify). Páginas por token têm `noindex`, `nofollow`.
- Estado de segurança e histórico: `../security/README.md`. **RLS pendente**: dados internos e buckets ainda abertos a contas de cliente até a migration `20261004000000` ser aplicada (`../security/rls-internal-only.md`).
- Verificação do esquema vivo: `src/integrations/supabase/types.ts` (gerado) lista 103 tabelas; 6 criadas por migrations foram removidas depois (`projeto_fases`, `influencer_approvals`, `email_flows`, `email_flow_enrollments`, `shared_calendar_connection`, `vault_access_requests`).

Módulos do app interno

Todos vivem sob `/time?section=<chave>` (exceto detalhes de cliente e projeto e o Chat V2). Cada módulo = uma `*Section.tsx` em `src/components/` + uma pasta de apoio. Permissão entre parênteses.

Seção	Função em uma linha
<code>inicio</code>	Painel pessoal: trabalho, agenda, mural, comentários, lembretes, clima
<code>clientes</code> (clientes)	Cadastro e ciclo de vida dos clientes
<code>campanhas</code> (campanhas)	Operação das campanhas, influenciadores e entregas
<code>projetos</code> (projetos)	Projetos e tarefas internas
<code>reunioes</code> (reunioes)	Agenda, disponibilidade, solicitações, Google Calendar
<code>comercial</code> (comercial)	Pipeline de leads, propostas, e-mail marketing
<code>financeiro</code> (financeiro)	Lançamentos, posição, análises
<code>time</code> (time)	Pessoas, carga, resposta, score operacional
<code>influenciadores</code> (influenciadores)	Banco de influenciadores
<code>metas</code> (metas)	Objetivos e indicadores
<code>chat</code> (chat)	Mensagens, canais, chamadas
<code>configuracoes</code> (configuracoes)	Perfil, workspace, permissões, integrações, segurança
<code>problemas</code>	Central de Problemas (reportar e triar bugs)
Marketing (dentro de Comercial/Marketing)	E-mail, blog, AEO, tráfego pago, editorial

Início

`InicioDashboard.tsx` (+ `components/inicio/`). Cards: **Meu trabalho** (tarefas via `task-aggregation`), **Agenda, Mural, Comentários, Lembretes** (`reminders.functions` , `personal_reminders`), cabeçalho climático (`weather-*` , `Open-Meteo`), diálogo "Bom dia" e avisos de release. Preferências por usuário em tempo real. É a **origem da linguagem visual** do design system.

Clientes

`ClientesSection` , `clientes/ClienteCard` · `ClienteFiltersBar` · `ClienteFormSheet` · `ClienteDetailPage` · `ClienteContratosSection` · `ClienteFinancicalSummary` · `PortalAccessSection` · `ClienteStatusControl` . - Status do cliente: Captação → Ativo → Encerrado → Arquivado (`validate_cliente_status` ; só admin arquiva). - Criar cliente chama `createClienteComOrganizacao` (cria a **organização** do portal junto). - Detalhe: contratos (`contratos.functions`), resumo financeiro, **acessos ao portal** (convites, papéis, campanhas liberadas — `client-access.functions` , `organization-invites`). - Vem de: conversão de Lead ou "Importar do CRM".

Campanhas

`CampanhasSection` (~1.700 linhas, lista + detalhe), `campanhas/*` (card, filtros, ativação, NPS, **página de inscrição**), `influenciadores/InfluencerBoard` (kanban/curadoria de influenciadores na campanha), `lib/entrega-engine` , `campanha-status` , `campanha-aprovacao` . - A campanha mora dentro de `clientes.data.campanhas` ; dados operacionais em tabelas `campanha_*` . - Fluxo do influenciador na campanha: inscrito → curadoria → enviado ao cliente → aprovado/recusado → entregas (roteiro/conteúdo/publicação) → resultados. - **NPS** do cliente (`campanha_nps`) e do influenciador (`campanha_nps_influenciador` , com avaliação interna). - **Inscrição pública** com token (`signupToken`) e perguntas personalizáveis.

Projetos e tarefas

`ProjetosSection` , `projeto.$id` (página única com seções por âncora), `tasks/TaskBoard` (workspace de tarefa: subtarefas, tags, dependências, bloqueios, tempo, comentários, anexos, rich-editor), `ProjectBugsPanel` . - Tarefas de **projeto, campanha e Marketing avulso** são tabelas diferentes agregadas em "Meu trabalho". - Bloqueio de tarefa com questionário (`task-blocks`); dependências não travam, só avisam. - Link público de bugs por projeto (`/bugs/$token`).

Reuniões

`ReunioesSection` , `meetings/*` (Agenda, Calendário, Disponibilidade, Solicitações, entrada por link), `UpcomingMeetingAlert` , `MeetingReminderToast` . Sincroniza com **Google Calendar** (`google-calendar.functions` , `OAuth`, cron diário). Pode anexar chamada interna (WebRTC).

Comercial

`ComercialSection` , `comercial/*` (`PipelineBoard` , `LeadDrawer` , `LeadCard` , `FollowUp` , `SimuladorPropostaDialog` , `convertLead`) , `comercial-engine` , `pricing` . - Etapas de oportunidade; realtime sobre `Leads` (AppShell notifica novo lead). - **Simulador de Proposta** (tiers × formatos) e **calculadora pública** por token. - `convertLeadToClienteEProjeto` : `lead` → `Cliente` (sempre em Captação) + `Projeto`; grava `crmLeadId` . - Ações automáticas por oportunidade (`runOpportunityAction`) , visões salvas, interações.

Marketing

`MarketingSection` + `marketing/*` : **E-mail** (campanhas, passos, públicos, templates, fluxos automáticos, dashboard; provedor Resend), **Blog** (editor markdown, publicação agendada, webhook para o site via Make, curtidas e comentários), **AEO Monitor** (prompts, rodadas, respostas por IA, evidências), **Tráfego pago**, **Editorial**.

Financeiro

`FinanceiroSection` com 3 áreas: **Resumo** (posição, requer atenção), **Lançamentos** (entradas/saídas, filtros, importação CSV, cobrança, marcar como pago), **Análises** (fluxo de caixa, despesas, receita por cliente, resultado por campanha). Lançamentos ligam a cliente/campanha/influenciador; pagamentos de influenciador alimentam o financeiro; contratos geram alertas de vencimento.

Time

`TimeV2Page` , `time-v2/*` , `team/*` : tabela de membros, carga, perfil (comunicação, tempo de resposta, entregas da semana, insights) e **Score Operacional** (`performance-engine` , config em Configurações). Tempo de resposta vem do Chat (`get_member_response_time`).

Influenciadores

`influenciadores-v2/InfluencerBancoV2Page` : banco com filtros, card, drawer e avaliação; dados bancários só com `influenciadores:bancario` . Alimenta campanhas e e-mail marketing (picker).

Metas

`metas/*` : **Objetivos** com **Indicadores** (peso, histórico, evolução, linha esperada), hoje atualizados **manualmente** (`dataSource` : "auto" está reservado para o futuro). Saúde: saudável, atenção, em risco, atrasado, concluído, não iniciado, cancelado — calculada só por `metas-engine` .

Chat

V1 (`ChatSection` , `chat-store`) e V2 (`chat-v2/*` , rotas `/chat-v2`). Canais públicos/privados, DMs, **@menções** de pessoa, tarefa, projeto, campanha e cliente (`mention-kinds`) , anexos, mensagens de voz, reações, fixadas/salvas, busca, threads, links com preview, **chamadas** (`call-controller` , WebRTC) e push.

Configurações

Seções: Geral/Workspace, Perfil, Preferências, Disponibilidade, Áudio e Vídeo, Segurança (MFA, cofre), **Time e permissões**, **Integrações** (webhook de leads, webhooks de saída, Google), Precificação, Score operacional, Dados e backup, **Auditoria**.

Central de Problemas

`problemas/*` : qualquer membro **reporta** (com contexto automático: tela, navegador, versão, captura) e comenta; quem tem `problemas` **tria** (status, prioridade, área, responsável, nota interna). Os reports do portal do cliente também chegam aqui.

Shell e transversais

`AppShell` : sidebar global (256/68px), topbar, busca global, notificações (menções, tarefas, mensagens), timer ativo, tema, aviso de versão. Transversais: `useConfirm` , `FilterToolbar` , `Kpi` , `PageHeader` , `EmptyState` , notificações/push, release notes.

Portal do cliente, links externos e integrações

Portal do cliente — três gerações

Rota	Estado	Acesso
/portal-v2/*	Atual, destino de todo login de cliente	Sessão Supabase + organização <code>client</code> ativa
/portal/\$token/*	Em uso, por link	Token público fixo do cliente, sem login
/portal-app/*	Legado: só redirects para /portal-v2	—

Portal V2 (`features/client-portal-v2` , rotas `portal-v2/*`)

Layout `PortalV2Shell` . Guard (`portal-v2/route.tsx`) : exige sessão, MFA resolvido, troca de senha se pendente, aceita convites pendentes, resolve o ambiente; e aplica o **gate de NPS** — se há NPS do mês pendente, bloqueia a navegação até responder (`nps-guard`).

Página	Função
Início	Resumo, atenção (o que espera o cliente), progresso das campanhas, atividade
Campanhas / <code>\$id</code>	Visão geral, creators , conteúdo, arquivos, timeline, resultados
Aprovações	Perfis de influenciador e entregas aguardando o cliente
Conteúdos, Arquivos, Relatórios	Entregas, anexos, relatórios mensais (URL assinada)
NPS	Avaliação mensal por campanha
Notificações, Perfil, Conta, Segurança	Conta e preferências
Configurações → Acessos	Administradores do cliente convidam e gerenciam pessoas (papéis e campanhas)

Ações do cliente no V2 (server functions `portal-auth.functions` , sufixo `Session`) : **responder influenciador** (aprovar/recusar), **responder entrega**, abrir **relatório** (URL assinada) e **NPS** (porta obrigatória). As demais funções `Session` (reabrir aprovação, editar briefing/observações/anexos, enviar demanda, reportar bug, artigos do blog) existem mas **nenhuma tela do V2 as chama** hoje; essas ações vivem no portal por token (`cliente-link.functions`). Papéis: `client_standard` (admin), `client_approver` (aprova), `client_viewer` (lê).

Portal por token (`portal.$token/*`)

Mesmo conteúdo essencial (início, campanhas, aprovações, relatórios, solicitações) sem login, via `cliente-link.functions` (as mesmas ações, sufixo sem `Session`). Visual próprio em `components/portal/*` . Idiomas via `portal-i18n` . `noindex` .

Links externos (por token, sem login)

URL	Quem usa	O que faz	Credencial
/inscricao/\$token	Influenciador	Inscribe-se numa campanha (redes, nicho, perguntas personalizadas, anexo); idempotente	<code>signupToken</code> da campanha
/nps-influenciador/\$token	Influenciador	Avalia a experiência (notas + "voltaria a trabalhar")	token em <code>campanha_nps_influenciador</code>
/bugs/\$token	Cliente/influenciador	Lista, reporta e marca bugs/sugestões do projeto (com print)	Token do projeto
/calculadora-proposta/\$token	Lead	Monta e salva uma proposta no simulador	Token do lead
/email/descadastro/\$token	Destinatário	Descadastro (entra em <code>email_unsubscribes</code> e cancela o destinatário em todas as campanhas)	<code>unsubscribe_token</code> do destinatário
/portal/\$token/*	Cliente	Portal sem login	Token do cliente
/criar-senha , /redefinir-senha	Cliente/equipe	Convite e recuperação	Link do Supabase Auth

Todas devolvem `noindex` , `nofollow` e escopam a leitura/escrita ao dono do token. Tentativas inválidas caem em estado de erro amigável.

Endpoints de máquina

Endpoint	Chamado por	Proteção	O que faz
POST /api/public/leads	Make / Typeform / formulários	X-Webhook-Secret (em webhook_settings); CORS	Insere o lead via service-role (o disparo lead.created / lead.won para webhooks de saída acontece em comercial.functions , ao criar/ganhar pela UI)
POST /api/webhooks/resend	Resend	Assinatura Svix (segredo whsec_ guardado pelo admin em email_provider_settings)	Atualiza o estado de cada envio em email_sends
GET /api/google/oauth-callback	Google	state em google_oauth_states ; APP_URL canônico	Conclui OAuth do Calendar
GET /api/cron/email-flows	Vercel Cron 0 13 * * *	Bearer CRON_SECRET	Avança fluxos de e-mail automáticos
GET /api/cron/google-calendar-sync	Vercel Cron 0 6 * * *	Bearer CRON_SECRET	Sincroniza reuniões ↔ Google

Integrações de saída

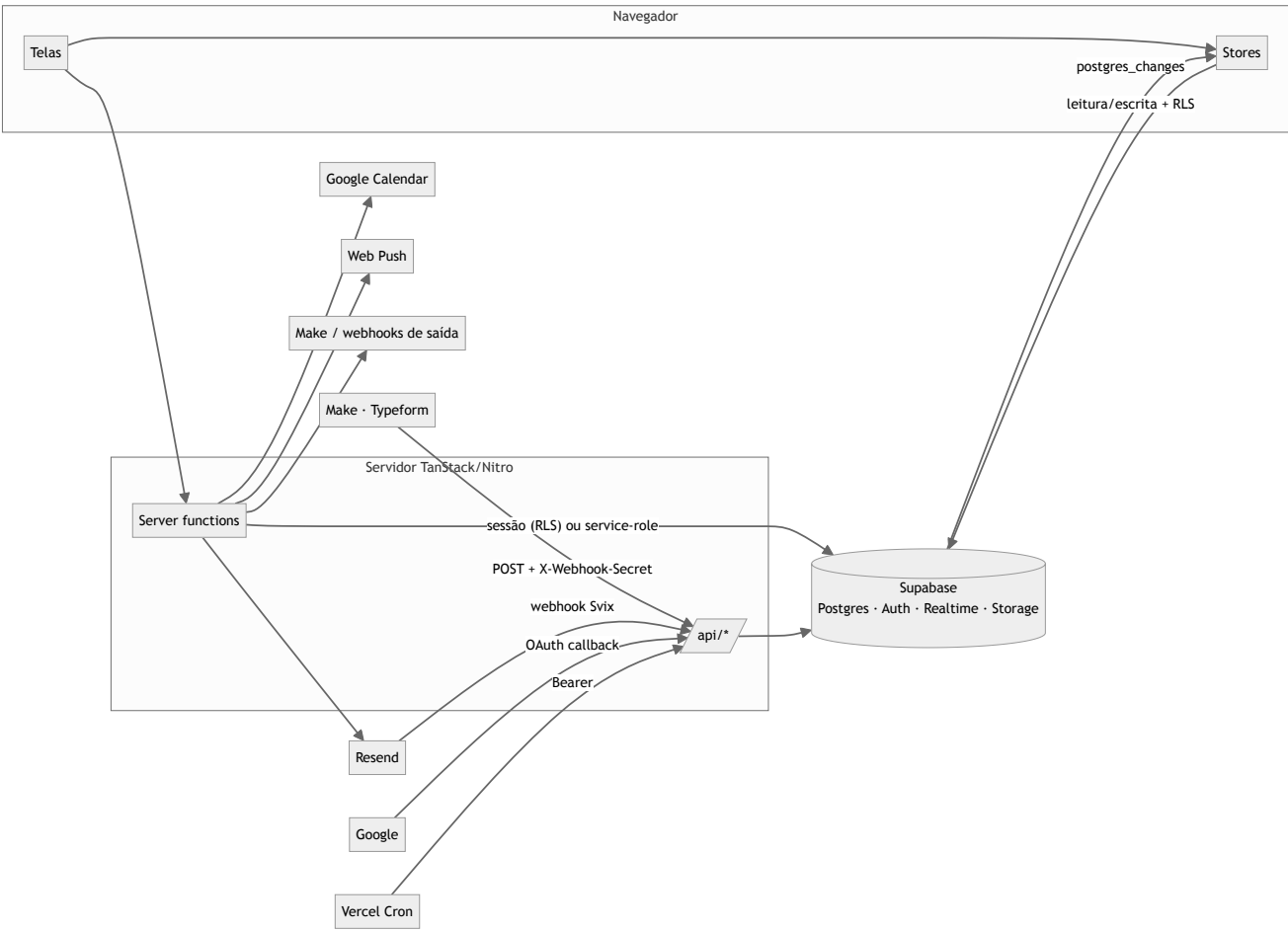
Destino	Como	Gatilho
Webhooks configuráveis (Make etc.)	outgoing_webhooks (admin, Configurações → Integrações), POST JSON {event,data,timestamp} ; best-effort	lead.created , lead.won
Webhook do blog (Make)	BLOG_WEBHOOK_URL , campo action = upsert/archive/delete	Ciclo de vida do artigo
Resend	email-provider.server , configurado em email_provider_settings	Disparos de campanha e fluxos
Google Calendar	OAuth + googleapis.com/calendar/v3	Reuniões
Web Push	VAPID (push.functions , push_subscriptions)	Chat, menções, lembretes
Open-Meteo	weather-service	Cabeçalho da Início
WhatsApp (wa.me), redes sociais	Links gerados na UI	Contato e perfis

Como as partes se comunicam e como as funcionalidades se ligam

1. Os canais de comunicação

Há **cinco** formas de uma parte falar com outra. Quase tudo passa pelo Supabase (banco como ponto de encontro); não há fila de mensagens nem API REST própria além dos endpoints `/api/*`.

Canal	Quem fala com quem	Como
Leitura/escrita direta	Navegador → Supabase	<code>supabase-js</code> com a sessão do usuário; RLS decide o que passa. É o caminho dos <code>stores</code> (<code>clientes</code> , <code>projetos</code> , <code>financeiro_lancamentos</code> , <code>chat</code> ...)
Server functions	Navegador → servidor → Supabase/serviços	<code>useServerFn(...)</code> ; token do usuário anexado automaticamente; usadas quando precisa de segredo, <code>service-role</code> , e-mail, regra sensível ou token público
Realtime	Supabase → navegador	<code>postgres_changes</code> em canais nomeados (<code>rt-*</code>); atualiza caches e dispara notificações
Webhooks/Endpoints	Sistema externo ↔ <code>/api/*</code>	Entrada: leads, Resend, Google. Saída: webhooks de leads e blog
Agendados	Vercel Cron → <code>/api/cron/*</code>	E-mail flows e Google Calendar, uma vez por dia

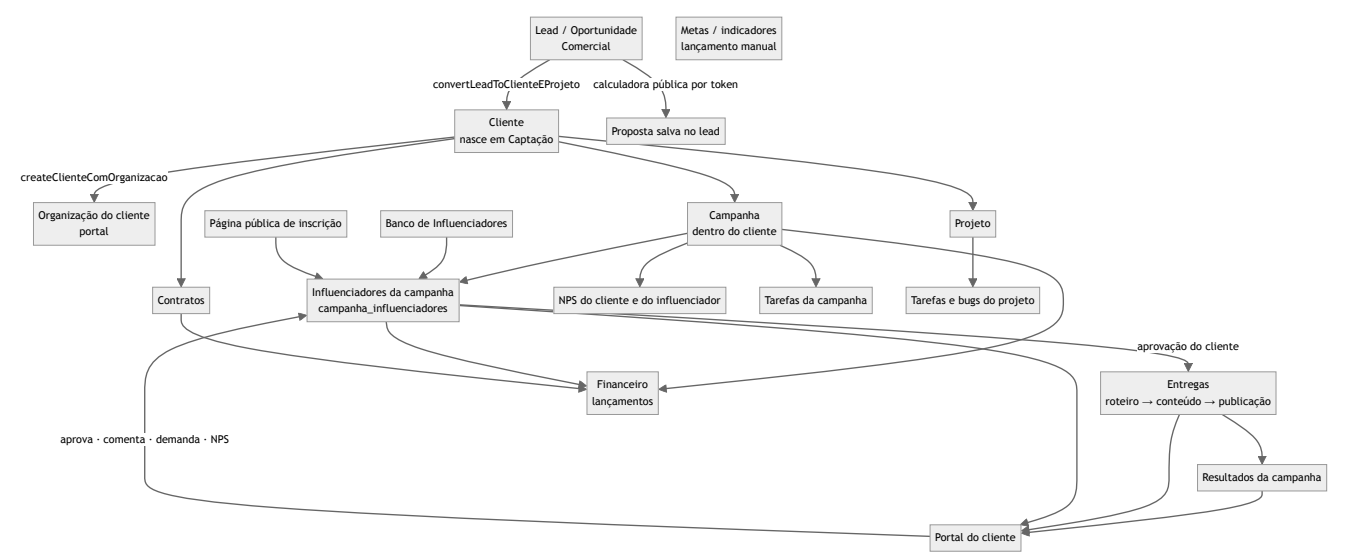


Tempo real: o que escuta o quê

Canal Realtime	Tabela / evento	Reação
<code>rt-leads-*</code> (Comercial)	<code>leads</code> *	Recarrega o pipeline
<code>rt-nav-leads-*</code> (AppShell)	<code>leads</code> INSERT	Notificação de novo lead
<code>rt-campanha-influenciadores-notify</code>	<code>campanha_influenciadores</code> UPDATE	Ação do cliente (aprovou/recusou/comentou) vira item no sino da equipe
<code>rt-campanha-tarefas-demand-notify</code>	<code>campanha_tarefas</code> INSERT	Demanda enviada pelo cliente vira notificação + toast

Canal Realtime	Tabela / evento	Reação
rt-chat-channels , rt-chat-messages , rt-chat-status	tabelas de chat	Mensagens, status, não lidas, menções
rt-portal-campanha-influenciadores , rt-portal-app-...	campanha_influenciadores	O portal do cliente reflete mudanças da equipe
shared_state:changes	shared_state	Estado compartilhado entre abas e pessoas
rt-dashboard-prefs-* , rt-bomdia-*	prefs/briefing	Início e "Bom dia"
rt-<tabela>-* (stores de lista)	clientes , projetos , reunioes , financeiro_lancamentos ...	Cache local sincronizado entre usuários
O cache local também guarda cópia em localStorage e o estado do time é consultado a cada 30 s.		

2. A cadeia principal do negócio



Leitura da cadeia: 1. **Lead** → **Cliente**. `convertLeadToClienteEProjeto` cria Cliente (sempre **Captação**, nunca Ativo direto), grava `crmLeadId` e cria um Projeto. A ativação é uma ação explícita separada (`ClienteStatusControl`). 2. **Cliente** → **organização**. Todo cliente novo ganha organização própria; é ela que dá acesso ao Portal V2 e isola dados. 3. **Cliente** → **campanha** → **influenciadores**. Influenciadores entram na campanha pelo **Banco**, por **inscrição pública** ou manualmente; passam por curadoria interna e vão ao cliente. 4. **Aprovação no portal**. O cliente aprova/recusa perfis e entregas; isso volta à equipe em tempo real (sino). 5. **Entregas** → **resultados**. Estado e métricas das entregas alimentam a aba de resultados da campanha. O **relatório mensal** é um arquivo guardado no bucket `relatorios-mensais` e entregue ao cliente por URL assinada (não é gerado a partir das entregas). 6. **Dinheiro**. Lançamentos podem referenciar cliente, campanha e influenciador; contratos geram alerta de vencimento; o financeiro por campanha compõe Análises. 7. **Metas** hoje são alimentadas por **lançamento manual** de indicadores (a fonte `auto` existe no modelo, mas está reservada para o futuro) e produzem saúde por `metas-engine` ; o **Time** consome tarefas, entregas e chat para carga, resposta e score.

3. Funcionalidades correlatas (quem toca em quem)

Funcionalidade	Está ligada a	Natureza do vínculo
Cientes	Comercial, Campanhas, Contratos, Financeiro, Portal, Chat (@cliente)	Conversão de lead; campanhas embutidas; organização; menções
Campanhas	Influenciadores (banco + inscrição), Entregas, NPS, Tarefas, Financeiro, Portal, Chat (@campanha, canal por campanha)	Mesmo <code>campanha_id</code> em tabelas <code>campanha_*</code>
Banco de Influenciadores	Campanhas, Financeiro (pagamento), E-mail marketing (picker), Avaliações	Reuso do perfil; dados bancários separados por permissão
Entregas	Portal (aprovação), Time (semana de entregas), Performance, Resultados	Estado calculado por <code>entrega-engine</code>
Comercial	Cientes, Projetos, E-mail marketing (leads), Webhooks de saída, Calculadora pública	Conversão, <code>lead.created</code> / <code>lead.won</code> , token de proposta
Projetos/Tarefas	Chat (@tarefa/@projeto), Time, Início (Meu trabalho), Problemas (bugs de projeto), Metas	<code>task-aggregation</code> junta projeto + campanha + marketing

Funcionalidade	Está ligada a	Natureza do vínculo
Reuniões	Google Calendar, Chat (chamada), Início (agenda), lembretes, push	OAuth + cron diário
Financeiro	Contratos, Campanhas, Influenciadores, Clientes	clienteId/campanhaId/influenciadorId no lançamento
Metas	Independente dos demais módulos (indicadores manuais; dataSource: auto reservado)	metas-engine
Time	Chat (tempo de resposta), Tarefas, Entregas, Configurações (score)	performance-engine , get_member_response_time
Chat	Todos (menções), Push, Início, Notificações	mention-kinds : pessoa, tarefa, projeto, campanha, cliente
Central de Problemas	Portal do cliente (reporta), Projetos (bugs), links /bugs/\$token , AppShell (botão), Hypito	Mesma tabela bug_reports
Notificações	Realtime de leads, ações do cliente, demandas, mensagens, release	Sino do AppShell + toast + push
Configurações	Permissões de todos os módulos, Integrações, Segurança, Score	profiles.permissions , outgoing_webhooks , webhook_settings
E-mail marketing	Leads, Clientes, Influenciadores (públicos), Resend (envio/webhook), Cron (fluxos), descadastro	email_* , unsubscribe_token
Blog	Webhook para o site (Make), Portal (cliente curte/comenta artigos)	blog_* , BLOG_WEBHOOK_URL
Hypito	Chat (hypito_payload), lembretes, briefing diário, relatórios, alertas, releases	Tabelas hypito_* [inferido: serviço externo]

4. Fluxos ponta a ponta

A. Cliente aprova um influenciador Portal V2 → respondCampanhaInfluSession → campanha_influenciadores.data (inclui lastClientAction) → Realtime UPDATE → AppShell mostra item no sino e atualiza o card da campanha; o portal também reflete via seu próprio canal.

B. Cliente envia uma demanda Portal **por token** → submitClientDemand (cliente-link.functions ; o V2 ainda não tem essa tela) → insere em campanha_tarefas → Realtime INSERT → notificação + toast na equipe, e a tarefa entra em "Meu trabalho".

C. Lead chega de um formulário Typeform/Make → POST /api/public/leads (segredo) → leads → Realtime → pipeline atualiza e AppShell notifica. Ao ganhar/criar pela UI, lead.won / lead.created saem para os webhooks configurados.

D. Inscrição de influenciador Link /inscricao/\$token → submitInscricaoCampanha (idempotente) → entra na campanha como "inscrito" → aparece na curadoria do Banco/Board → pode ser enviado ao cliente.

E. E-mail marketing Admin cria campanha de e-mail → activateCampaign → cron diário avança fluxos → Resend envia → webhook Svix atualiza email_sends → descadastro (unsubscribe_token) cancela o destinatário em todas as campanhas.

F. Reunião ↔ Google Conexão OAuth (startGoogleOAuth → callback) → sync manual (syncAllMeetingsToGoogle / importGoogleEventsToMeetings) e cron google-calendar-sync diário.

G. Convite de pessoa do cliente Admin do cliente (Portal → Configurações → Acessos) ou equipe (Cliente → Acessos ao portal) → server function de convite (link gerado por auth.admin.generateLink , enviado por sendEmail /Resend) → /criar-senha → login → acceptPendingInvites → ambiente client → /portal-v2/inicio . Gate de NPS pode bloquear até responder.

H. Release scripts/release.ts publish → platform_releases → Realtime → VersionWatcher /release notes avisam; public/version.json sem cache indica bundle novo.

5. Onde a comunicação é frágil (a observar)

- Muita regra depende de **JSONB** (clientes.data.campanhas , *.data): vínculos não têm chave estrangeira; consistência é por convenção e triggers (sync_scoped_table_data_id).
- Realtime sem fila**: se a aba estiver fechada, a notificação só chega via push ou ao recarregar.
- Dois chats** (V1 e V2) e **três gerações de portal** convivem; mudanças precisam considerar ambos.
- Webhooks de saída são best-effort**: falha é só logada.

Auditoria técnica — 2026-10-03

Mapear → auditar → classificar → priorizar → implementar → validar. Nada foi reescrito "por sensação": cada mudança abaixo tem uma medição ou evidência de código. O que **não** foi possível verificar está dito na seção 12.

Atualização (2026-10-03, depois da auditoria): os jogos (Termo/Zip, card "Pausa rápida" da Início) foram removidos do app. Onde este relatório os menciona (lazy-load, `games/zip/fixture.ts`), refere-se ao estado anterior à remoção. As tabelas de jogos permanecem no banco, sem uso.

1. Método e limites

- Medido no build:** tamanho de chunks e fechamento de imports estáticos (`performance/README.md`).
- Análise estática:** grafo de imports (arquivos não alcançáveis, ciclos), migrations em ordem (políticas efetivas, índices, funções `SECURITY DEFINER`), varredura de server functions, `bun audit` .
- Validação:** `tsc` , ESLint (0 erros), 808 testes, build — rodados depois de cada bloco.
- Não feito:** nenhuma consulta ao banco vivo, nenhum teste em produção/logado, nenhum teste de carga. Portanto RLS e índices são **achados estáticos** a confirmar (ver 12).

2. Inventário e classificação

Camada	Arquivos	Linhas	Papel
<code>src/components</code>	358 (50 pastas)	~99.500	UI dos módulos + shell + <code>ui/</code> (shadcn) + <code>shared/</code> (canônicos)
<code>src/lib</code>	202	~39.700	regra de negócio, stores, engines puros, 34 arquivos <code>*.functions.ts</code> (161 server functions)
<code>src/routes</code>	74	~9.200	rotas (app, portal V2, portal por token, páginas públicas, <code>/api/*</code>)
<code>src/features/client-portal-v2</code>	83	~9.300	portal do cliente
Banco	140 migrations · 103 tabelas (live, <code>types.ts</code>) · 37 funções <code>SECURITY DEFINER</code>		Supabase
Testes	65 arquivos · 808 testes		engines, regras puras, parte da UI
Dependências	54 runtime · 19 dev		

Classificação (por inspeção do código, não por nome): - **CORE:** `AppShell` , `_authenticated/route` , stores (`*-store.ts` , `table-array-store` , `shared-sync`), `integrations/supabase` , `permissions` , `user-environment` . - **MÓDULOS:** Início, Clientes, Campanhas, Projetos/Tarefas, Reuniões, Comercial, Financeiro, Time, Influenciadores, Metas, Chat, Marketing, Configurações, Problemas. - **INFRAESTRUTURA:** `*.functions.ts` , `*.server.ts` , `routes/api/*` , crons, webhooks, e-mail (Resend), push, Google Calendar, rate limit, audit log. - **COMPARTILHADO:** `ui/*` , `shared/*` (PageHeader, Kpi, FilterToolbar, EmptyState, SectionCard...), `design-tokens` , `use-confirm` . - **LEGADO (ainda ligado):** Chat V1 (`ChatSection`) ao lado do V2; portal por token ao lado do V2; `/portal-app/*` (só redirects); `/foco` , `/time-v2` , `/banco-influenciadores-v2` (redirects). - **EXPERIMENTAL:** `/design-system` , `/design-system-finance-concept` (vitrine interna). - **CÓDIGO MORTO / DUPLICADO:** ver seção 7.

3. Documentação

Antes: 26 arquivos Markdown espalhados (relatórios soltos, pasta `platform/` que duplicava o Design System, comandos repetidos). Depois: estrutura por assunto, índice único (`docs/README.md`), ADRs.

Documento	Decisão	O que foi feito
<code>CLAUDE.md</code>	ATUALIZAR	Ponteiros e regras de camada; bullets obsoletos corrigidos
<code>AGENTS.md</code>	MANTER	—
<code>.lovable/plan.md</code>	MANTER (gerenciado pelo Lovable)	Registrado no índice como histórico, não movido
<code>docs/security-audit-report.md</code> , <code>security-remediation-plan.md</code>	MANTER (histórico)	Movidos para <code>security/</code> com data no nome
<code>docs/codebase-optimization-report.md</code>	ARQUIVAR	Movido para <code>performance/</code> com data no nome

Documento	Decisão	O que foi feito
docs/design-system/* (+ technical/)	MANTER	Imagens de validação reunidas em validation/
docs/platform/01...06,08	MANTER	Movidos para architecture/ ; 04 virou modules/README.md
docs/platform/02 (comandos, env, deploy, convenções)	CONSOLIDAR	Extraído para development/guia.md
docs/platform/07-DESIGN.md	CONSOLIDAR → REMOVER	Duplicava o contrato; parte única foi para design-system/APLICACAO-E-MIGRACAO.md
docs/platform/README.md	REMOVER	Substituído por docs/README.md
src/routes/README.md	MANTER	Convenção de rotas, junto do código
Novos		docs/README.md , development/guia.md , security/README.md , security/rls-internal-only.md , performance/README.md , decisions/0001-0003 , esta auditoria

4. Design System

Fonte de verdade: ../design-system/DESIGN-SYSTEM.md . Estado atual medido no código:

Divergência	Antes desta rodada	Agora
text-[9px] , text-[10px] , text-[13px]	~105 arquivos	0
font-bold / font-light	37 arquivos	10 usos (KPI lead e casos documentados)
window.confirm	5	2 (fluxos síncronos)
<select> nativo com estilo próprio	~111	1 (interno do date-field); resto em NativeSelect
Card sem borda sobre canvas cinza (PageCanvas)	9 módulos	0
Resumos de KPI como sistemas visuais separados	7+ componentes	Kpi canônico (SummaryStat / PageSummaryPanel removidos)
Paleta direta de cor (amber-500 , emerald-500 ...)	—	~690 usos em ~80 arquivos (maior dívida restante)
<table> cru	11	11
Radix Select	5 arquivos	5

Consistência sem homogeneização: cada módulo mantém sua arquitetura funcional (Kanban do Comercial, workspace de tarefas, chat...), compartilhando shell, cabeçalho, tipografia, KPI, filtros e estados.

5. Performance (frontend e dados)

Ver ../performance/README.md . Resumo medido: carga inicial comum **-47% gzip**; landing do app **-41% gzip**; o gargalo era uma dependência lib → componente que arrastava 7.100 linhas de UI e o recharts para o bundle inicial.

Dados: 16 stores baixam tabelas inteiras antes de qualquer tela (P1, não alterado — contrato síncrono get() exige auditar consumidores); ~26 canais realtime; polling de 15-60 s em 8 pontos, parte redundante com realtime; 53 select("*") . Chat bem indexado (convo_id , created_at ; GIN em menções). Sem N+1 óbvio nas server functions auditadas.

6. Banco de dados

- **Schema vivo x migrations:** types.ts (103 tabelas) bate com as migrations menos 6 tabelas removidas depois — sem drift.
- **RLS:** todas as tabelas têm RLS; 10 sem policy (só service-role) por desenho; **achado P1:** policies authenticated USING (true) e storage por bucket_id expõem dado interno a contas de cliente → migration proposta, **não aplicada**.
- **Funções:** 37 SECURITY DEFINER ; 1 sem search_path fixo (enforce_cliente_archive_admin_only) → corrigida na migration pendente. RPCs expostas ao cliente conferidas (guardas por auth.uid() / is_internal_team_member).
- **Índices:** organization_members só tem UNIQUE(organization_id, user_id) e é consultada por user_id dentro das policies (is_internal_team_member) → índice parcial na migration. A lista automática de "FK sem índice" (60 colunas) é **majoritariamente colunas de auditoria** (updated_by) e tem falsos positivos de parser; não foi tratada.
- **Regra:** nada destrutivo; migration idempotente com rollback documentado.

7. Código morto e duplicação

Grafo de imports (733 arquivos): só 6 não alcançáveis a partir das rotas.

Item	Classe	Ação
<code>src/lib/portal-auth.functions 2.ts</code> (cópia antiga, menos segura)	SEGURO REMOVER	Removido
<code>components/financeiro/CobrancaDialog.tsx</code> (sem importadores desde a simplificação do Financeiro)	SEGURO REMOVER	Removido (histórico no git)
<code>requestPasswordReset</code> (endpoint legado sem sessão)	SEGURO REMOVER	Removido
<code>lib/access-guards.server.ts</code> , <code>lib/hypito-permissions.server.ts</code>	REVISAR	Mantidos (infra de autorização nunca ligada)
<code>lib/games/zip/fixture.ts</code> , <code>lib/semver.ts</code>	NÃO REMOVER	Usados por testes
10 funções <code>*Session</code> do portal V2 sem chamadores	REVISAR	Mantidas; decisão de produto
~63 exports com uma única ocorrência	REVISAR	Lista por varredura; não removidos sem confirmar referências dinâmicas
Duplicações de UI (SummaryStat/PageSummaryPanel/PipelineSummary, 5 barras de filtro, 49 selects)	CONSOLIDADO	<code>Kpi</code> , <code>FilterToolbar</code> , <code>NativeSelect</code>

8. Arquitetura

- **Camadas:** `lib` importava componentes (20+ módulos, 5 ciclos). Tratado o pior caso (modelo de influenciadores e pessoas/motivos de tarefa → ADR 0001). Restam 19 imports `lib` → `components` (14 `import type`; 5 de valor: `campanhaStatus` x3 e `clienteStatus` de `campanha-ui` / `cliente-ui`, e os tipos de `marketing-tasks`).
- **Ciclos restantes** (3, de 2 arquivos cada): `TaskBoard` ↔ `TaskActivityPanel` (mesmo workspace), `PendingNpsGate` ↔ `PortalV2Shell`, `routeTree.gen` ↔ `router` (gerado). Eram 5 (incluía `InfluencerBoard` ↔ `chat-store` ↔ `permissions` ↔ `projetos` e `InicioDashboard` ↔ `cartões`, resolvidos).
- **Arquivos gigantes:** `InfluencerBoard` ~6.000, `TaskBoard` ~5.400, `ChatSection` ~2.800, `AppShell` ~2.250, `portal-widgets` ~2.100.
- **UI → banco:** 2 chamadas `supabase.from` diretas em componentes (resto passa por stores/server functions): aceitável.

9. UX e complexidade estrutural

Mapeadas e **parcialmente** tratadas: filtro-dentro-de-filtro (Financeiro, Comercial, Problemas → padrão único), controles de período como "coleção de filtros" (→ unidade de contexto), drawers sobrecarregados (Comercial reorganizado). Pendentes: abas aninhadas em Projeto → E-mails → campanha; inscrição de campanha (overlay com 3 abas e preview); `TaskBoard` (workspace com muitas camadas); Chat V1+V2 simultâneos.

10. Padrão entre módulos

Todos compartilham shell, `PageHeader`, tipografia oficial, `surface-card`, KPI e filtros canônicos. Diferenças legítimas por função preservadas (Kanban, workspace de tarefa, chat, calendário). Validação visual logada ainda pendente (ver 12).

11. Observabilidade

Existe: captura de erros do host (Lovable) + `error-capture / error-page`, 113 `console.error/warn`, `access_audit_log` e `settings_audit_log`, Central de Problemas (reports com diagnóstico), `platform_releases`. Não existe: log estruturado nem alerta de falha de **cron/webhook/e-mail** (só `console`). Recomendação sem ferramenta nova: registrar o resultado de cada execução de cron e de cada falha de webhook em tabela própria e expor na Central de Problemas (P3).

12. Riscos restantes e o que não foi verificado

1. **RLS pendente** (P1): confirmar no banco vivo e aplicar `20261004000000` — `rls-internal-only.md`.
2. **Stores por inteiro no startup** (P1): precisa de dado real de volume para decidir a divisão.
3. **Lazy-load** de `TaskDialog` e jogos validado por build e testes, **não** no navegador logado.
4. **Rate limit** de login consultivo e escrita pública por token sem limite (P2).
5. Validação visual logada dos módulos migrados (claro/escuro, mobile, telas densas).
6. **Regra aparentemente estranha, só documentada:** o `rate limit` de login é chamado pelo cliente antes do `signInWithPassword` (consultivo); `shared_state` permite `UPDATE` de qualquer linha a qualquer autenticado (tratado na migration, não no código); funções `*Session` do portal V2 sem tela. Nenhuma foi "corrigida" por conta própria.
7. Um teste (`projeto-ui.test.ts`) dependia do dia UTC e falhava entre 21h e 00h (BRT); corrigido o **teste** (o código do app já usa Brasília). Nenhuma regra de negócio alterada.

13. Matriz de prioridades

#	Problema	Área	Impacto	Risco	Esforço	Prio	Solução	Status
1	Policies/storage abertos a contas de cliente (relatórios de outros clientes, precificação, estado compartilhado)	Segurança	Alto (vazamento entre clientes)	Médio (testar com dados reais)	Baixo	P1	Migration 20261004000000	Pronta, pendente de aplicar
2	16 stores baixam tudo antes de qualquer tela	Performance/escala	Alto com crescimento	Alto (contrato síncrono)	Alto	P1	Init lazy por módulo	Documentado
3	UI de 7.100 linhas + recharts no bundle inicial	Performance	Alto	Baixo	Médio	P1	Extrair modelo para lib/	Feito (–47% gz comum)
4	Workspace de tarefas (600 KB) no shell	Performance	Alto	Baixo	Médio	P1	TaskDialog lazy + task-people	Feito
5	Endpoint legado sem sessão (requestPasswordReset)	Segurança	Médio (spam a admins)	Baixo	Baixo	P2	Remover	Feito
6	logLoginFailure sem teto	Segurança	Médio	Baixo	Baixo	P2	Rate limit	Feito
7	Erros de banco crus em endpoints públicos (18)	Segurança	Médio	Baixo	Baixo	P2	throwSafeDbError	Feito
8	Rate limit de login consultivo; escrita pública sem limite	Segurança	Médio	Médio	Médio	P2	Limites no servidor por token/IP	Documentado
9	Cores por paleta direta (~690 usos)	Design System	Médio (inconsistência)	Baixo	Alto	P2	Mapear status → tokens	Documentado
10	Arquivos gigantes e lib → components residual	Arquitetura	Médio	Médio	Alto	P2	Dividir por responsabilidade; mover tipos	Parcial (ADR 0001)
11	Índice em organization_members(user_id)	Banco	Baixo hoje, cresce	Baixo	Baixo	P2	Índice parcial	Na migration pendente
12	Auth: init sequencial (3 idas)	Performance	Baixo-médio	Baixo	Baixo	P2	Promise.all preservando a ordem de decisão	Feito
13	Teste dependente de fuso (falhava 21h–00h)	Dívida técnica	Baixo	Nulo	Baixo	P2	Usar todayIsoInBrasilia	Feito
14	vitest /ESLint rodavam em worktrees antigos	Dívida técnica	Médio (lint/test quebrados)	Nulo	Baixo	P2	Excluir .claude	Feito
15	Dependências com advisory (dompurify)	Segurança	Baixo	Baixo	Baixo	P3	bun update dompurify	Feito; demais são só de build
16	10 funções *Session sem chamador; guardas nunca ligados	Dívida técnica	Baixo	Baixo	Baixo	P3	Decisão de produto	Documentado
17	Polling redundante com realtime	Performance	Baixo	Baixo	Médio	P3	Consolidar	Documentado
18	Sem log de falha de cron/webhook	Observabilidade	Baixo	Baixo	Médio	P3	Tabela de execuções	Documentado
19	11 <table> cruas, 5 Radix Select, 2 window.confirm	Design System	Baixo	Baixo	Médio	P3	Migrar	Documentado
20	target="_blank" sem rel	Segurança	Nulo	Nulo	—	—	Verificado por parser JSX: todas as tags têm rel	Falso positivo

14. Resumo

Segurança — encontrados: exposição de dado interno e de outros clientes por RLS/storage abertos a contas de cliente; endpoint sem sessão legado; log de auditoria inundável; erros de banco vazando; advisory em dompurify. Corrigidos no código: endpoint legado, rate limit, erros, dompurify. **Restante:** aplicar a migration (pendente de conferência no banco vivo), rate limits do servidor para escrita pública.

Performance — gargalo: dependência lib → UI e workspace de tarefas no shell. Medido: carga inicial comum –49% bruto / –47% gzip; landing do app –43% / –41%. Pendente: stores completas no startup.

Arquitetura — camada lib → components corrigida nos casos de maior custo; ADR 0001; ciclos 5 → 3 (um gerado); arquivos gigantes documentados.

Design System — 9/10/13px, selects, canvases, confirmações, KPI e filtros consolidados; paleta direta, tabelas cruas e Radix Select pendentes.

Documentação — 26 arquivos reorganizados em 7 áreas, 1 índice, 3 ADRs, 1 duplicado dissolvido.

Código — 3 remoções seguras (arquivo duplicado, diálogo órfão, endpoint legado); 2 extrações de módulo; lint e testes voltaram a rodar por padrão.

Banco/Supabase — 1 migration (policies, storage, `search_path`, índice) com rollback e consultas de verificação; sem mudança destrutiva; nada aplicado.

15. Próximos passos que valem a pena

1. Conferir e aplicar a migration de RLS (P1).
2. Validar visualmente os módulos logado e o diálogo de tarefa/jogos.
3. Decidir a carga lazy das stores com volume real de dados (P1).
4. Mapear status → tokens e eliminar a paleta direta (P2).
5. Rate limit de escrita pública no servidor (P2).

Riscos e lacunas conhecidos

Atualizado em 2026-10-03 (após a auditoria técnica). Detalhes e matriz de prioridades: `auditoria-2026-10.md`.

O que não foi verificado

- **Banco vivo:** a análise de RLS/índices é estática (migrations + `types.ts`); nada foi executado contra produção. Consultas de verificação em `../security/rls-internal-only.md`.
- **Dados reais:** volume e qualidade dos registros não foram consultados (afeta a decisão sobre carga das stores).
- **Telas autenticadas:** o app exige login; o visual foi validado só em harness isolado, e o comportamento de lazy-load (diálogo de tarefa) só por build, não no navegador logado.
- **Hypito:** o serviço não está neste repositório; comportamento deduzido de tabelas e permissões.
- **Funções de servidor:** autorização e validação de entrada conferidas por varredura (middleware, zod, rate limit); o corpo de cada uma não foi revisado.
- **Portal por token:** conteúdo de cada tela não lido em detalhe.

Riscos ativos

Risco	Efeito	Onde tratar
RLS aberta a contas de cliente em dados internos e buckets (migration pendente)	Cliente lê dados de outros clientes e da agência pela API direta	<code>security/rls-internal-only.md</code> (P1)
16 stores carregadas por inteiro antes de qualquer tela	Carga inicial cresce com os dados	<code>performance/README.md</code> (P1)
Vínculos em JSONB sem FK (<code>clientes.data.campanhas</code> , <code>*,data</code>)	Inconsistência silenciosa	triggers existentes + testes; avaliar normalizar
Rate limit de login consultivo; escrita pública por token sem rate limit	Abuso com token vazado / força bruta direta no Auth	<code>security/README.md</code> (P2)
Realtime sem fila	Notificação perdida com aba fechada	push + recarga cobrem parte
Sem CI	Regressão chega ao deploy	rodar <code>typecheck</code> , <code>lint</code> , <code>test</code> , <code>build</code> antes de cada push
Arquivos enormes (<code>InfluencerBoard</code> ~6.000, <code>TaskBoard</code> ~5.400, <code>ChatSection</code> ~2.800, <code>AppShell</code> ~2.250)	Difícil manter e testar	dividir por responsabilidade
Service-role em server functions	Bypass de RLS se faltar checagem	manter validação de token/ <code>assertAdmin</code> em toda função nova
Webhooks de saída best-effort	Evento perdido sem aviso	<code>log/retry</code>
Validação visual logada pendente	Contraste ou separação ruins em telas densas	validar módulo a módulo

Pendências conhecidas do produto

- `membros` e `senhas` são permissões **decorativas** (ver `dados-e-acesso.md`).
- `clientes` aceita a permissão `campanhas` por causa do JSONB.
- Chat V1 e V2, e portal V2 + token + legado, convivem.
- Metas: indicadores automáticos (`dataSource: auto`) reservados, sem implementação.
- **10 funções *Session do portal V2 sem nenhum chamador** (`reopenCampanhaInfluSession` , `updateInfluBriefingSession` , `updateInfluObservacoesSession` , `updateInfluBriefingAnexoSession` , `submitClientDemandSession` , `submitPortalBugReportSession` , `submitRelatorioNpsSession` , `loadArtigoEngagementSession` , `toggleArtigoLikeSession` , `addArtigoComentarioSession`): o portal V2 hoje só usa aprovar/recusar influenciador, responder entrega, relatórios e NPS; solicitações, bug report e artigos existem só no portal por token. São endpoints autenticados (superfície sem uso) — confirmar com o produto se o V2 deve ganhar essas telas ou se as funções saem.
- `access-guards.server.ts` / `hypito-permissions.server.ts` : guardas de autorização nunca ligados.
- `.claude/scheduled_tasks 2.lock` solto na raiz (sobra de ambiente; não versionado).